

A Tableau Abstraction of Clifford+ T Circuits

Abstract

We define an abstract interpretation of Clifford+ T quantum circuits. The abstract state is a Clifford tableau together with a list of Pauli *axes*, one for each T gate. A Clifford gate updates the tableau exactly. A T gate is not tracked exactly: it leaves the tableau unchanged and records its axis. We give the concretization of an abstract state as a set of unitaries, and prove soundness: the unitary of every circuit lies in the concretization of its abstract state. The development follows the Lean formalization in `PauliFold/Tableau.lean` of the separate `lean-paulifold` repository; Appendix A lists the correspondence.

1 Preliminaries

States and operators. A register of n qubits has state space $(\mathbb{C}^2)^{\otimes n} \cong \mathbb{C}^{2^n}$, with the computational basis $\{|x\rangle : x \in \{0, 1\}^n\}$. Operators on it are the complex $2^n \times 2^n$ matrices $M_{2^n}(\mathbb{C})$. A matrix U is *unitary* if $UU^\dagger = U^\dagger U = I$; we write $U(2^n)$ for the unitary matrices. For a unitary U and an operator A , the *conjugate* of A by U is $UAU^\dagger$. Conjugation by a unitary preserves products,

$$U(AB)U^\dagger = (UAU^\dagger)(UBU^\dagger), \quad (1)$$

because $U^\dagger U = I$.

Pauli matrices. The single-qubit Pauli matrices are

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

They satisfy $X^2 = Y^2 = Z^2 = I$ and

$$XY = iZ, \quad YZ = iX, \quad ZX = iY, \quad YX = -iZ, \quad ZY = -iX, \quad XZ = -iY. \quad (2)$$

In particular $Y = iXZ$. Two letters $a, b \in \{I, X, Y, Z\}$ commute ($ab = ba$) if one of them is I or they are equal, and anticommute ($ab = -ba$) otherwise.

Definition 1.1 (Pauli strings). A *Pauli string* on n qubits is a matrix

$$P = i^k P_1 \otimes P_2 \otimes \cdots \otimes P_n, \quad k \in \mathbb{Z}_4, \quad P_j \in \{I, X, Y, Z\}.$$

We call i^k its *phase* and P_j its *letter at qubit j* . The set of Pauli strings is $\mathcal{P}_n$. For a qubit q and a letter L , we write L_q for the string with letter L at q , I elsewhere, and phase 1. In particular X_q and Z_q are X and Z on qubit q .

Lemma 1.2 (Products of Pauli strings). *The product of two Pauli strings is a Pauli string. Explicitly, $(i^k \otimes_j P_j)(i^l \otimes_j Q_j) = i^{k+l} \otimes_j (P_j Q_j)$, and each $P_j Q_j$ is a letter times a power of i , by (2).*

Proof. The tensor product satisfies $(\otimes_j A_j)(\otimes_j B_j) = \otimes_j A_j B_j$. $\square$

Lemma 1.3 (Commutation). *Two Pauli strings P and Q either commute or anticommute. They commute exactly when the number of qubits j at which P_j and Q_j anticommute is even.*

Proof. By Lemma 1.2, PQ and QP have the same letters, and the phase of PQ is that of QP times (-1) for each qubit where the letters anticommute. $\square$

Lemma 1.4 (Decomposition). *Every Pauli string is a product of single-qubit strings:*

$$i^k P_1 \otimes \cdots \otimes P_n = i^k (P_1)_1 (P_2)_2 \cdots (P_n)_n.$$

Proof. By Lemma 1.2: the factors act on different qubits, and each has I at every other qubit. $\square$

Gates and circuits. We use the gates of Table 1. A single-qubit gate G on qubit q acts as G on q and as the identity on the other qubits.

gate	matrix	kind
H	$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$	Clifford
X, Z	X, Z	Clifford
S, S [†]	diag(1, i), diag(1, $-i$)	Clifford
CX _{c,t}	$ x\rangle \mapsto x\rangle$ with x_t replaced by $x_t \oplus x_c$	Clifford
CZ _{c,t}	$ x\rangle \mapsto (-1)^{x_c x_t} x\rangle$	Clifford
T, T [†]	diag(1, ω), diag(1, $\bar{\omega}$), with $\omega = e^{i\pi/4}$	rotation

Table 1: The Clifford+T gate set. $c \neq t$ for the two-qubit gates.

A *circuit* is a finite sequence of gates $g_1 g_2 \cdots g_m$, applied in that order. Its *semantics* is the unitary

$$\llbracket g_1 \cdots g_m \rrbracket = G_m \cdots G_2 G_1,$$

where G_j is the matrix of g_j . So $\llbracket \varepsilon \rrbracket = I$ for the empty circuit, and $\llbracket c g \rrbracket = G \llbracket c \rrbracket$.

Clifford gates map Pauli strings to Pauli strings; T does not. For every Clifford gate G and every $P \in \mathcal{P}_n$, the conjugate $G^\dagger P G$ is again a Pauli string. For example, $H^\dagger X H = Z$. The rotations are different:

$$T X T^\dagger = \frac{1}{\sqrt{2}}(X + Y),$$

which is not a Pauli string. Our abstraction treats the Clifford gates exactly and the rotations approximately. It rests on one identity.

Lemma 1.5 (Rotations are combinations of I and Z). *Let R be T or T[†] on qubit q . Then there are $a, b \in \mathbb{C}$ with*

$$R = a I + b Z_q.$$

For T, $a = \frac{1+\omega}{2}$ and $b = \frac{1-\omega}{2}$. For T[†], replace ω by $\bar{\omega}$.

Proof. $aI + bZ = \text{diag}(a + b, a - b)$, and $\text{diag}(1, \omega)$ has this form exactly when $a + b = 1$ and $a - b = \omega$. On n qubits, the identity and Z_q both act as the identity away from q . $\square$

2 The abstract domain

The abstract state follows the circuit as it is processed from left to right. It separates the Clifford gates seen so far, which are tracked exactly, from the rotations, which are only remembered by their axes.

Definition 2.1 (Abstract states). An *abstract state* $\sigma = (x, z, \mathbf{A})$ consists of:

- rows $x_q, z_q \in \mathcal{P}_n$ for each qubit $q \in \{1, \dots, n\}$;
- a finite list $\mathbf{A} = (A_1, \dots, A_m)$ of Pauli strings, the *axes*.

The *initial state* is $\sigma_0 = (X, Z, ())$: that is, $x_q = X_q$ and $z_q = Z_q$ for every q , and there are no axes.

Definition 2.2 (Rows represent a Clifford). The rows of σ *represent* a unitary C if, for every qubit q ,

$$x_q = C^\dagger X_q C \quad \text{and} \quad z_q = C^\dagger Z_q C.$$

Intuitively, C is the Clifford part of the circuit processed so far. The row z_q is the Pauli string that C turns into Z_q at the output, written in the coordinates of the circuit's input. We say it is Z_q *pulled back* to the input.

Definition 2.3 (Pull-back). For an abstract state σ and a Pauli string $P = i^k P_1 \otimes \dots \otimes P_n$, define

$$\beta_\sigma(P) = i^k \rho_1(P_1) \rho_2(P_2) \dots \rho_n(P_n),$$

where

$$\rho_q(I) = I, \quad \rho_q(X) = x_q, \quad \rho_q(Z) = z_q, \quad \rho_q(Y) = i x_q z_q.$$

By Lemma 1.2, $\beta_\sigma(P)$ is a Pauli string.

Intuition for β . Think of a circuit as having an *input side* and an *output side*. A Pauli string P is a question one can ask at the output: an observable that can be measured there. The pull-back $\beta_\sigma(P)$ is the same question, asked at the input:

- *Measurement.* Measuring P after the Clifford C gives the same outcome statistics as measuring $C^\dagger P C$ before it (Lemma 2.4). So β_σ translates observables from output coordinates to input coordinates.
- *Gate by gate.* Each Clifford gate G at the end of the circuit changes how an output observable looks at the input: P at the output is $G^\dagger P G$ just before G . β_σ is the accumulated effect of all these translations.
- *Why $2n$ rows suffice.* Every Pauli string is a product of the generators X_q and Z_q (Lemma 1.4, with $Y = iXZ$), and conjugation preserves products. So it is enough to store the translations of the $2n$ generators; those are the rows. β_σ extends them to every Pauli string by multiplication.

Lemma 2.4 (The pull-back is conjugation). *If the rows of σ represent a unitary C , then for every $P \in \mathcal{P}_n$,*

$$\beta_\sigma(P) = C^\dagger P C.$$

Proof. First the single-qubit case, $\rho_q(L) = C^\dagger L_q C$:

- for I, $C^\dagger I C = I$;
- for X and Z, this is the definition of representing C ;
- for Y, $Y = i X Z$ gives $C^\dagger Y_q C = i (C^\dagger X_q C)(C^\dagger Z_q C) = i x_q z_q$ by (1).

Then, writing $P = i^k (P_1)_1 \cdots (P_n)_n$ (Lemma 1.4) and applying (1) (with $C^\dagger$ in the role of U),

$$C^\dagger P C = i^k (C^\dagger (P_1)_1 C) \cdots (C^\dagger (P_n)_n C) = i^k \rho_1(P_1) \cdots \rho_n(P_n) = \beta_\sigma(P). \quad \square$$

Example 2.5 (Pull-backs through H and CX).

1. *One qubit*, $C = H$. Since $H^\dagger X H = Z$ and $H^\dagger Z H = X$, the rows are $x = Z$ and $z = X$. So $\beta(Z) = X$: a Z measurement after H is an X measurement before it. For Y,

$$\beta(Y) = i x z = i Z X = i \cdot i Y = -Y,$$

matching $HYH = -Y$.

2. *Two qubits*, $C = CX_{1,2}$. The rows are

$$x_1 = X_1 X_2, \quad x_2 = X_2, \quad z_1 = Z_1, \quad z_2 = Z_1 Z_2.$$

So $\beta(Z_2) = Z_1 Z_2$: the Z value of the target at the output is the parity of both wires at the input. And $\beta(X_1) = X_1 X_2$: an X on the control at the output is an X on both wires at the input. Products follow by multiplication, for example

$$\beta(Z_1 Z_2) = z_1 z_2 = Z_1 \cdot Z_1 Z_2 = Z_2.$$

3 Abstract transformers

Definition 3.1 (Transformer of a Clifford gate). Let g be a Clifford gate. The new state $\sigma' = (x', z', A)$ has the same axes as σ , and rows given by the *row update* column of Table 2. Rows not mentioned there are unchanged. On the right of each update, x and z are the rows of σ , before the update.

gate	row update	pulled-back generators
H on q	swap x_q and z_q	$H^\dagger X_q H = Z_q, \quad H^\dagger Z_q H = X_q$
X on q	$z'_q = -z_q$	$X^\dagger Z_q X = -Z_q$
Z on q	$x'_q = -x_q$	$Z^\dagger X_q Z = -X_q$
S on q	$x'_q = -i x_q z_q$	$S^\dagger X_q S = -Y_q = -i X_q Z_q$
$S^\dagger$ on q	$x'_q = i x_q z_q$	$S X_q S^\dagger = Y_q = i X_q Z_q$
$CX_{c,t}$	$x'_c = x_c x_t, \quad z'_t = z_c z_t$	$X_c \mapsto X_c X_t, \quad Z_t \mapsto Z_c Z_t$
$CZ_{c,t}$	$x'_c = x_c z_t, \quad x'_t = z_c x_t$	$X_c \mapsto X_c Z_t, \quad X_t \mapsto Z_c X_t$

Table 2: The Clifford transformer. The middle column is the update. The right column gives the conjugates $G^\dagger X_q G$ and $G^\dagger Z_q G$ that justify it (Lemma 3.2), where they differ from X_q and Z_q .

Each update multiplies a few rows together. The rule is read off the right column: for example $CX^\dagger X_c CX = X_c X_t$, so the new x_c is the product of the rows for X_c and X_t . The next lemma states this in general.

Lemma 3.2 (Row updates are pull-backs). *Let g be a Clifford gate with matrix G . If the rows of σ represent a unitary C , the new rows are the old pull-backs of the conjugated generators:*

$$x'_q = \beta_\sigma(G^\dagger X_q G), \quad z'_q = \beta_\sigma(G^\dagger Z_q G) \quad \text{for every qubit } q.$$

Proof. By Lemma 2.4, $\beta_\sigma(P) = C^\dagger P C$. So by (1), β_σ turns a product of generators into the product of their rows.

- *Changed rows.* Apply this to the right column of Table 2. For example:
 - $CX_{c,t}$: $\beta_\sigma(X_c X_t) = x_c x_t$;
 - S : $\beta_\sigma(-i X_q Z_q) = -i x_q z_q$;
 - H : $\beta_\sigma(Z_q) = z_q$, so the new x_q is the old z_q .

Each result is the entry in the middle column.

- *Unchanged rows.* For a generator the table does not mention, $G^\dagger X_r G = X_r$ (or $G^\dagger Z_r G = Z_r$), and $\beta_\sigma(X_r) = x_r$.

The identities in the right column are the standard Clifford conjugation rules. Each is checked by multiplying the matrices of Table 1. $\square$

Example 3.3 (Applying the Clifford transformer: S then H). On one qubit, start from σ_0 and apply S , then H .

1. *After S .* The update is $x' = -i x z$, so

$$x' = -i X Z = -i \cdot (-i Y) = -Y, \quad z' = Z,$$

using $XZ = -iY$ from (2).

2. *After H .* Swap the rows: $x = Z$ and $z = -Y$.

Lemma 3.2 predicts these rows. The Clifford part is $C = HS$, and

$$C^\dagger X C = S^\dagger (H X H) S = S^\dagger Z S = Z, \quad C^\dagger Z C = S^\dagger (H Z H) S = S^\dagger X S = -Y.$$

So an X measured after the circuit is a Z at its input, and a Z at the output is $-Y$ at the input.

Example 3.4 (Applying the Clifford transformer: preparing a Bell pair). On two qubits, apply H on qubit 1, then $CX_{1,2}$.

after	x_1, x_2	z_1, z_2
start	X_1, X_2	Z_1, Z_2
H on 1	Z_1, X_2	X_1, Z_2
$CX_{1,2}$	$Z_1 X_2, X_2$	$X_1, X_1 Z_2$

The CX updates are $x'_1 = x_1 x_2 = Z_1 \cdot X_2$ and $z'_2 = z_1 z_2 = X_1 \cdot Z_2$.

The pull-backs of the Bell pair's two stabilizers are

$$\beta(X_1 X_2) = x_1 x_2 = Z_1 X_2 \cdot X_2 = Z_1, \quad \beta(Z_1 Z_2) = z_1 z_2 = X_1 \cdot X_1 Z_2 = Z_2.$$

So measuring $X_1 X_2$ or $Z_1 Z_2$ at the output is measuring Z_1 or Z_2 at the input. On the input $|00\rangle$ both are $+1$, which is why the output is the Bell state stabilized by $X_1 X_2$ and $Z_1 Z_2$.

Definition 3.5 (Transformer of a rotation). Let g be T or $T^\dagger$ on qubit q . The new state keeps the rows and appends the current row z_q to the axes:

$$\sigma' = (x, z, A \cdot z_q).$$

The axis z_q is the rotation's own axis Z_q pulled back to the input. The axes are stored in input coordinates, so later gates never need to update them.

Example 3.6 (Applying the rotation transformer). Continue Example 3.3 with a T : the circuit is SHT . The T keeps the rows $x = Z$, $z = -Y$ and records the axis $z = -Y$.

after	x	z	axes
start	X	Z	()
S	$-Y$	Z	()
H	Z	$-Y$	()
T	Z	$-Y$	($-Y$)

The T rotates about Z at its own position. Pulled back through HS , that is $-Y$ at the input. Only commutation with an axis matters, so the sign is irrelevant.

Definition 3.7 (Abstract semantics). Write $g^\sharp$ for the transformer of gate g . The abstract semantics of a circuit is

$$\llbracket g_1 \cdots g_m \rrbracket^\sharp = g_m^\sharp(\cdots g_2^\sharp(g_1^\sharp(\sigma_0)) \cdots).$$

4 Concretization

An abstract state describes a circuit through *facts* of the form “the circuit conjugates B into P ”.

Definition 4.1 (Facts). The *facts* of an abstract state $\sigma = (x, z, A)$ are the pairs

$$F(\sigma) = \{ (\beta_\sigma(P), P) : P \in \mathcal{P}_n \text{ and } \beta_\sigma(P) \text{ commutes with every } A \in A \}.$$

A unitary U *satisfies* the fact (B, P) if $UBU^\dagger = P$.

Definition 4.2 (Concretization). The *concretization* of σ is the set of unitaries satisfying all of its facts:

$$\gamma(\sigma) = \{ U \in U(2^n) : UBU^\dagger = P \text{ for every } (B, P) \in F(\sigma) \}.$$

More facts give a smaller set, and so a more precise description of the circuit. Proposition 6.1 below characterizes $\gamma(\sigma)$ directly.

Example 4.3 (Facts and concretization of SHT). Take the final state of Example 3.6: rows $x = Z$, $z = -Y$, and axis $-Y$.

The facts. Test each Pauli P : is $\beta(P)$ commuting with the axis $-Y$?

P	$\beta(P)$	commutes with $-Y$?	fact
I	I	yes	(I, I)
X	$x = Z$	no	—
Z	$z = -Y$	yes	($-Y, Z$)
Y	$ixz = iZ(-Y) = -X$	no	—

For Y we used $ZY = -iX$. Scalar multiples of these give the remaining facts. So the abstraction knows one thing: the circuit maps $-Y$ to Z .

Membership in γ . A unitary U is in γ exactly when $U(-Y)U^\dagger = Z$.

- $U = \text{THS}$, the circuit itself, is in γ :

$$S(-Y)S^\dagger = X, \quad HXH = Z, \quad T Z T^\dagger = Z.$$

Theorem 5.1 guarantees this in general.

- $U = \text{HS}$, the Clifford part alone, is in γ too, by the first two steps. The abstraction cannot see the T on this fact.
- $U = H$ is not in γ : $H(-Y)H = Y \neq Z$.

5 Soundness

Theorem 5.1 (Soundness). *For every Clifford+T circuit c ,*

$$\llbracket c \rrbracket \in \gamma(\llbracket c \rrbracket^\#).$$

That is: let $\sigma = \llbracket c \rrbracket^\#$ and $U = \llbracket c \rrbracket$. For every Pauli string P whose pull-back $\beta_\sigma(P)$ commutes with all the axes of σ ,

$$U \beta_\sigma(P) U^\dagger = P.$$

The proof maintains the following invariant along the circuit.

Definition 5.2 (Invariant). A unitary U and an abstract state $\sigma = (x, z, A)$ satisfy $\text{Inv}(U, \sigma)$ if there are unitaries C and V with:

- (i) $U = CV$;
- (ii) the rows of σ represent C ;
- (iii) $VB = BV$ for every Pauli string B that commutes with every axis in A .

Here C is the Clifford part of the circuit. V collects the rotations, conjugated into input coordinates. By (iii), V is invisible to every Pauli string that commutes with all the axes.

Lemma 5.3 (Initial state). $\text{Inv}(I, \sigma_0)$.

Proof. Take $C = V = I$. The rows $X_q = I^\dagger X_q I$ and $Z_q = I^\dagger Z_q I$ represent I , and I commutes with everything. $\square$

Lemma 5.4 (Clifford step). *Let g be a Clifford gate with matrix G . If $\text{Inv}(U, \sigma)$, then*

$$\text{Inv}(GU, g^\#(\sigma)).$$

Proof. Let $U = CV$ as in the invariant, and take $C' = GC$ and $V' = V$.

- (i): $GU = (GC)V = C'V'$.
- (ii): C' is unitary, as a product of unitaries. By Lemma 3.2 and then Lemma 2.4, for the new rows,

$$x'_q = \beta_\sigma(G^\dagger X_q G) = C'^\dagger G^\dagger X_q G C' = (GC)^\dagger X_q (GC) = C'^\dagger X_q C',$$

and likewise $z'_q = C'^\dagger Z_q C'$.

- (iii): the axes and V are unchanged. $\square$

Lemma 5.5 (Rotation step). *Let g be T or $\mathsf{T}^\dagger$ on qubit q , with matrix R . If $\text{Inv}(U, \sigma)$, then*

$$\text{Inv}(RU, g^\sharp(\sigma)).$$

Proof. Let $U = CV$ as in the invariant. By Lemma 1.5, $R = aI + bZ_q$. Since the rows represent C ,

$$C^\dagger RC = aC^\dagger C + bC^\dagger Z_q C = aI + bz_q. \quad (3)$$

Take $C' = C$ and $V' = (C^\dagger RC)V$.

- (i): $RU = RCV = C(C^\dagger RC)V = C'V'$, using $CC^\dagger = I$.
- (ii): the rows are unchanged, and still represent $C' = C$. V' is unitary, as a product of unitaries.
- (iii): let B commute with every axis of $g^\sharp(\sigma)$, that is, with z_q and with every $A \in \mathbf{A}$. Then B commutes with V , by the invariant for σ . It also commutes with $aI + bz_q$, since it commutes with I and with z_q . So by (3), B commutes with $V' = (aI + bz_q)V$. $\square$

Lemma 5.6 (Runs). *For every circuit c , $\text{Inv}(\llbracket c \rrbracket, \llbracket c \rrbracket^\sharp)$.*

Proof. By induction on the length of c . The base case is Lemma 5.3. For cg , we have $\llbracket cg \rrbracket = G \llbracket c \rrbracket$ and $\llbracket cg \rrbracket^\sharp = g^\sharp(\llbracket c \rrbracket^\sharp)$, so the step is Lemma 5.4 or Lemma 5.5. $\square$

Proof of Theorem 5.1. By Lemma 5.6, $U = \llbracket c \rrbracket = CV$, where the rows of $\sigma = \llbracket c \rrbracket^\sharp$ represent C and V satisfies (iii). Let $(B, P) \in F(\sigma)$. By Lemma 2.4, $B = \beta_\sigma(P) = C^\dagger PC$, and by the definition of facts, B commutes with every axis. By (iii), $VB = BV$, and so

$$UBU^\dagger = CVBV^\dagger C^\dagger = CBVV^\dagger C^\dagger = CBC^\dagger = CC^\dagger PCC^\dagger = P. \quad \square$$

Example 5.7 (The invariant along SHT). Follow the witnesses C and V of Lemmas 5.4 and 5.5 through the circuit of Example 3.6.

after	$U = \llbracket \cdot \rrbracket$	C	V
start	I	I	I
S	S	S	I
H	HS	HS	I
T	THS	HS	$aI + b(-Y)$

Here $a = \frac{1+\omega}{2}$ and $b = \frac{1-\omega}{2}$, as in Lemma 1.5.

- *The Clifford gates* only change C .
- *The T* leaves C and sets $V = C^\dagger \mathsf{T} C = aI + bC^\dagger ZC = aI + bz$, which is (3) with $z = -Y$.
- *Condition (i)* holds: $CV = \mathsf{HS}(\mathsf{HS})^\dagger \mathsf{T} \mathsf{HS} = \mathsf{THS}$.
- *Condition (iii)* holds: $V = aI - bY$ commutes with I and Y , exactly the Paulis that commute with the axis $-Y$.

The final step of the proof then recovers the fact of Example 4.3: $U(-Y)U^\dagger = C(-Y)C^\dagger = Z$.

6 What the concretization contains

The concretization has a direct description in terms of the Clifford C and the axes.

Proposition 6.1 (Characterization). *Let $\sigma = \llbracket c \rrbracket^\sharp$ for a circuit c . Let C be a Clifford unitary represented by the rows of σ ; for example, the product of the Clifford gates of c , in order. Let K be the set of Pauli strings that commute with every axis of σ . Then*

$$\gamma(\sigma) = \{ CW : W \in \text{U}(2^n) \text{ and } WB = BW \text{ for every } B \in K \}.$$

Proof. Conjugation by a Clifford is a bijection on $\mathcal{P}_n$, so as P ranges over $\mathcal{P}_n$, the pull-back $\beta_\sigma(P) = C^\dagger PC$ ranges over all of $\mathcal{P}_n$. The facts are therefore exactly the pairs $(B, CBC^\dagger)$ for $B \in K$. For a unitary U , put $W = C^\dagger U$, so that $U = CW$. Then

$$UBU^\dagger = CBC^\dagger \iff WBW^\dagger = B \iff WB = BW,$$

where the first step multiplies on the left by $C^\dagger$ and on the right by C , and the second uses $W^\dagger W = I$. $\square$

Corollary 6.2 (Clifford circuits are exact). *If c contains no rotations, then $\gamma(\llbracket c \rrbracket^\sharp) = \{ \lambda \llbracket c \rrbracket : |\lambda| = 1 \}$.*

Proof. With no axes, $K = \mathcal{P}_n$. The Pauli strings span $M_{2^n}(\mathbb{C})$, so a W commuting with all of them is a scalar; a unitary scalar has modulus 1. And the Clifford C of the proposition is $\llbracket c \rrbracket$. $\square$

Computing the concretization. To compute $\gamma(\sigma)$ in examples, we need the matrices W allowed by Proposition 6.1. For a Pauli string Q with phase 1, record which qubits carry an X part and which carry a Z part:

$$v(Q) = (u, w) \in \mathbb{F}_2^n \times \mathbb{F}_2^n, \quad u_j = [Q_j \in \{X, Y\}], \quad w_j = [Q_j \in \{Z, Y\}].$$

Up to phase, products of strings correspond to sums of vectors. By Lemma 1.3, Q and Q' commute exactly when $\langle v(Q), v(Q') \rangle = \sum_j (u_j w'_j + w_j u'_j) = 0$ in $\mathbb{F}_2$.

Lemma 6.3 (Products of axes). *Let $S \subseteq \mathbb{F}_2^{2n}$ be spanned by the vectors of the axes, and let K be the set of Pauli strings that commute with every axis. A matrix W commutes with every $B \in K$ if and only if*

$$W = \sum_{Q: v(Q) \in S} w_Q Q$$

for some coefficients $w_Q \in \mathbb{C}$. That is, W is a combination of products of axes, where Q ranges over the phase-1 strings.

Proof. The 4^n phase-1 Pauli strings form a basis of $M_{2^n}(\mathbb{C})$, so every W is uniquely $\sum_Q w_Q Q$. For a Pauli string B , $BQB^\dagger = \pm Q$, with the sign $-$ exactly when $\langle v(B), v(Q) \rangle = 1$. So W commutes with B if and only if $w_Q = 0$ for every Q that anticommutes with B .

Now K is the set of B with $v(B) \in S^\perp$, the complement under $\langle \cdot, \cdot \rangle$. So W commutes with all of K if and only if $w_Q = 0$ unless $v(Q) \in (S^\perp)^\perp$. The form $\langle \cdot, \cdot \rangle$ is nondegenerate, so $(S^\perp)^\perp = S$. $\square$

With Proposition 6.1, this gives

$$\gamma(\sigma) = \left\{ CW : W \in \text{U}(2^n), \ W \in \text{span}\{ Q : v(Q) \in S \} \right\}. \quad (4)$$

In words: the circuit is its Clifford part C , times an unknown unitary built only from the axes. With no axes, $S = \{0\}$ and W is a scalar, which recovers the corollary.

For instance, in Example 4.3 the only axis is $-Y$, so S is spanned by $v(Y)$ and

$$\gamma = \{ \text{HS}(uI + vY) \text{ unitary} \}.$$

This contains the circuit, $\text{THS} = \text{HS}(aI - bY)$ (Example 5.7), and its Clifford part HS (take $u = 1, v = 0$). It does not contain H : the matrix $(\text{HS})^\dagger \text{H} = \text{S}^\dagger$ is diagonal, and not of the form $uI + vY$.

7 Worked examples

Each example gives the circuit and the abstract state after each gate, then the facts and the concretization. A fact (B, P) with P replaced by $i^k P$ becomes $(i^k B, i^k P)$, so we list facts only for phase-1 strings P . Facts are also closed under products, so it is enough to list them for P ranging over the generators X_q and Z_q . On one qubit we drop the qubit index.

Example 7.1 (The empty circuit). Here $\sigma_0 = (x = X, z = Z, ())$, so $\beta(P) = P$ for every P , and there are no axes. The facts are (P, P) for every P , and

$$\gamma = \{ \lambda I : |\lambda| = 1 \}.$$

The abstraction is exact up to a global phase.

Example 7.2 (One Hadamard).

after	x	z	axes
start	X	Z	()
H	Z	X	()

The facts include (Z, X) and (X, Z) : H maps Z to X and X to Z . With no axes, $\gamma = \{ \lambda \text{H} \}$. Clifford circuits are abstracted exactly.

Example 7.3 (A CNOT). For $\text{CX}_{1,2}$ the rows are those of Example 2.5(2), with no axes. The facts include $(X_1 X_2, X_1)$, (X_2, X_2) , (Z_1, Z_1) and $(Z_1 Z_2, Z_2)$, which is exactly the standard description of the CNOT by its action on Paulis. Then $\gamma = \{ \lambda \text{CX}_{1,2} \}$.

Example 7.4 (One T).

after	x	z	axes
start	X	Z	()
T	X	Z	(Z)

The rows are unchanged, so $\beta(P) = P$. A fact needs P to commute with the axis Z , so the facts are (I, I) and (Z, Z) . There is no fact about X : a T gate turns X into $(X + Y)/\sqrt{2}$, which is not a Pauli string. Here S is spanned by $v(Z)$, so by (4)

$$\gamma = \{ aI + bZ \text{ unitary} \} = \{ \text{diag}(u, v) : |u| = |v| = 1 \}.$$

The abstraction knows that the circuit is diagonal, and nothing more. It cannot tell T from S , Z or $T^\dagger$.

Example 7.5 (T then $T^\dagger$). Both rotations record the axis Z , so the axes are (Z, Z) and S is still spanned by $v(Z)$. The concretization is again the set of diagonal unitaries. The circuit $\mathbb{T}\mathbb{T}^\dagger$ is the identity, but the abstraction does not know it: axes record where rotations act, not their angles. This is the price of the abstraction, and the reason an optimizer must merge such pairs separately.

Example 7.6 (H, T, H).

after	x	z	axes
start	X	Z	()
H	Z	X	()
T	Z	X	(X)
H	X	Z	(X)

The T acts on Z at its own position, and Z there is X at the input. So it records the axis $z = X$. At the end, $C = HH = I$ and $\beta(P) = P$.

- *Facts:* (X, X) only, since Z anticommutes with the axis X .
- *Concretization:* by (4), $\gamma = \{aI + bX \text{ unitary}\}$. This contains $HTH = \frac{1+\omega}{2}I + \frac{1-\omega}{2}X$.

Example 7.7 (T then H : axes stay in input coordinates).

after	x	z	axes
start	X	Z	()
T	X	Z	(Z)
H	Z	X	(Z)

The H after the T changes the rows but not the axis, which stays Z at the input. Now $\beta(X) = Z$ commutes with the axis and $\beta(Z) = X$ does not.

- *Facts:* (Z, X) . The circuit HT sends Z to X , because T fixes Z and then H turns it into X .
- *Concretization:* $\gamma = \{HW : W \text{ diagonal unitary}\}$.

Example 7.8 (A T after a CNOT: information survives the rotation).

after	x_1, x_2	z_1, z_2	axes
start	X_1, X_2	Z_1, Z_2	()
$CX_{1,2}$	X_1X_2, X_2	Z_1, Z_1Z_2	()
T on 2	X_1X_2, X_2	Z_1, Z_1Z_2	(Z_1Z_2)

The T on qubit 2 acts on the output parity Z_2 , which is Z_1Z_2 at the input. Check each generator against the axis Z_1Z_2 :

- $\beta(X_1) = X_1X_2$ anticommutes with Z_1Z_2 at two qubits, so it commutes. Fact: (X_1X_2, X_1) .
- $\beta(X_2) = X_2$ anticommutes with Z_1Z_2 . No fact.
- $\beta(Z_1) = Z_1$ commutes. Fact: (Z_1, Z_1) .
- $\beta(Z_2) = Z_1Z_2$ commutes. Fact: (Z_1Z_2, Z_2) .

So three of the four generators are still known exactly after the rotation. In particular the fact (X_1X_2, X_1) involves an X , even though a T has acted. The concretization is

$$\gamma = \{CX_{1,2}(aI + bZ_1Z_2) \text{ unitary}\},$$

and it contains the circuit, since $T_2 CX_{1,2} = CX_{1,2}(CX_{1,2}^\dagger T_2 CX_{1,2})$ and $CX_{1,2}^\dagger T_2 CX_{1,2} = \frac{1+\omega}{2}I + \frac{1-\omega}{2}Z_1Z_2$.

Remark 7.9 (Scope).

- *Other rotations.* The only property of T and $\mathsf{T}^\dagger$ used is Lemma 1.5, that they are combinations of I and Z_q . The same proof covers every Z -rotation $R_z(\theta) = \text{diag}(e^{-i\theta/2}, e^{i\theta/2})$.
- *Folding.* The analysis says nothing about merging rotations; it only approximates the circuit's unitary. An optimizer may use the facts to justify rewrites, such as moving a rotation past gates it commutes with, but that is a separate step.
- *Not covered.* The development handles no Toffoli gates, measurements or resets. Branches and loops are the subject of Section 8.

8 Branches and loops

So far the analysis runs over a single circuit. Programs with nondeterministic branches and loops need one more operation: the *join* of two abstract states, which must describe the unitaries of either branch.

Programs. Programs are generated by

$$p ::= g \mid p; q \mid \text{if } \star \text{ then } p \text{ else } q \mid \text{while } \star \text{ do } p,$$

where g ranges over the gates of Table 1 and $\star$ is an unknown condition. The *collecting semantics* $\llbracket p \rrbracket \subseteq \mathsf{U}(2^n)$ is the set of unitaries p can implement:

- $\llbracket g \rrbracket = \{G\}$;
- $\llbracket p; q \rrbracket = \{VU : U \in \llbracket p \rrbracket, V \in \llbracket q \rrbracket\}$;
- $\llbracket \text{if } \star \text{ then } p \text{ else } q \rrbracket = \llbracket p \rrbracket \cup \llbracket q \rrbracket$;
- $\llbracket \text{while } \star \text{ do } p \rrbracket$ is the smallest set that contains I (zero iterations) and contains VU whenever it contains U and $V \in \llbracket p \rrbracket$.

8.1 What happens to axes in a join

Consider $\text{if } \star \text{ then } \mathsf{T}_q \text{ else skip}$. Both branches start from the same state. The first records the axis z_q ; the second records nothing. A fact about the program must hold whichever branch ran.

- In the first branch, a fact (B, P) needs B to commute with z_q .
- In the second branch, it needs nothing extra.

So the facts of the program are those whose B commutes with the axes of *both* branches. The joined state therefore carries the *union* of the two lists of axes.

This makes sense because all axes are written in the same coordinates: the input coordinates of the point where the branches split. Two branches may end with different rows, but their axes can always be compared directly.

Rows are different. They describe the Clifford part at the end of each branch, and if the branches end with different Clifford parts, the joined state cannot use either branch's rows. The simplest precise answer is to keep a row when the two branches agree on it and to forget it otherwise.

8.2 Partial states

Definition 8.1 (Partial states). A *partial state* $\sigma = (x, z, \mathbf{A})$ is an abstract state whose rows may be *unknown*: $x_q, z_q \in \mathcal{P}_n \cup \{?\}$. The domain also contains a top element $\top$, standing for

“nothing is known”.

- *Pull-back.* $\beta_\sigma(P)$ is defined by Definition 2.3 when every row it uses is known, and is undefined otherwise. It uses the row x_q when $P_q \in \{X, Y\}$, and z_q when $P_q \in \{Z, Y\}$.
- *Facts.* The facts are the pairs $(\beta_\sigma(P), P)$ with $\beta_\sigma(P)$ defined and commuting with every axis.
- *Concretization.* $\gamma(\sigma)$ is as in Section 4, and $\gamma(\top) = U(2^n)$.

Definition 8.2 (Transformers on partial states).

- *Clifford gate.* Apply Table 2, with the rule that a product involving $?$ is $?$. For example, $CX_{c,t}$ sets $x'_c = x_c x_t$, which is $?$ if x_c or x_t is $?$.
- *Rotation on q .* If z_q is known, append it to the axes, as in Definition 3.5. If $z_q = ?$, the rotation’s axis is unknown and the result is $\top$.
- $\top$ is mapped to $\top$.

Definition 8.3 (Join and order). The *join* of two partial states is

$$(x, z, A) \sqcup (x', z', A') = (x \sqcap x', z \sqcap z', A \cdot A'), \quad (x \sqcap x')_q = \begin{cases} x_q & \text{if } x_q = x'_q, \\ ? & \text{otherwise,} \end{cases}$$

and likewise for z . Joining with $\top$ gives $\top$. The *order* $\sigma \sqsubseteq \sigma'$ holds when either:

- $\sigma' = \top$; or
- every known row of σ' is the same row of σ , and every axis of σ is an axis of σ' .

Both σ and σ' lie below $\sigma \sqcup \sigma'$.

Definition 8.4 (Abstract semantics of programs).

- *Gates:* $\llbracket g \rrbracket^\sharp(\sigma) = g^\sharp(\sigma)$.
- *Sequencing:* $\llbracket p; q \rrbracket^\sharp = \llbracket q \rrbracket^\sharp \circ \llbracket p \rrbracket^\sharp$.
- *Branching:* $\llbracket \text{if } \star \text{ then } p \text{ else } q \rrbracket^\sharp(\sigma) = \llbracket p \rrbracket^\sharp(\sigma) \sqcup \llbracket q \rrbracket^\sharp(\sigma)$.
- *Loops.* For **while** $\star$ **do** p , starting from σ , iterate

$$I_0 = \sigma, \quad I_{k+1} = I_k \sqcup \llbracket p \rrbracket^\sharp(I_k),$$

and return the first I_{k+1} with $\llbracket p \rrbracket^\sharp(I_{k+1}) \sqsubseteq I_{k+1}$. If none is found within $2n + 3$ steps, return $\top$.

The order is decidable, so the check is effective. Soundness does not depend on the bound: if the check never passes, the loop returns $\top$, which is sound. The bound is chosen because each join can only forget rows, of which there are $2n$, and once the rows stop changing the body adds no axes that are not already recorded. This makes the check pass quickly in practice, but we do not prove that it always does within the bound.

8.3 Soundness

A partial state is justified by a *completion*: a full state that fills in its unknown rows.

Definition 8.5 (Completions and soundness). A state τ with all rows known *completes* a partial state σ when:

- every known row of σ is the same row of τ ; and
- every axis of τ is an axis of σ .

We write $\text{Sound}(U, \sigma)$ when some completion τ of σ satisfies $\text{Inv}(U, \tau)$. $\text{Sound}(U, \top)$ always holds.

Lemma 8.6 (Facts through completions). *If $\text{Sound}(U, \sigma)$, then $U \in \gamma(\sigma)$.*

Proof. Let τ be the completion. If $\beta_\sigma(P)$ is defined, it is computed from known rows, which are also rows of τ , so $\beta_\sigma(P) = \beta_\tau(P)$. If it commutes with every axis of σ , it commutes in particular with every axis of τ . The last step of the proof of Theorem 5.1 then gives $U \beta_\sigma(P) U^\dagger = P$. $\square$

Lemma 8.7 (Steps, order and join). *Let g be a gate with matrix G .*

- (a) *If $\text{Sound}(U, \sigma)$, then $\text{Sound}(GU, g^\sharp(\sigma))$.*
- (b) *If $\text{Sound}(U, \sigma)$ and $\sigma \sqsubseteq \sigma'$, then $\text{Sound}(U, \sigma')$.*
- (c) *If $\text{Sound}(U, \sigma)$ or $\text{Sound}(U, \sigma')$, then $\text{Sound}(U, \sigma \sqcup \sigma')$.*

Proof. Let τ complete σ , with $\text{Inv}(U, \tau)$.

- (a) *Clifford gate.* Each known new row of $g^\sharp(\sigma)$ is a product of known old rows. These equal the corresponding rows of τ , so it equals the new row of $g^\sharp(\tau)$. Neither state's axes change, so $g^\sharp(\tau)$ completes $g^\sharp(\sigma)$, and $\text{Inv}(GU, g^\sharp(\tau))$ holds by Lemma 5.4.
Rotation. If $z_q = ?$ the result is $\top$. Otherwise z_q is the row z_q of τ , so both states record the same axis, and Lemma 5.5 applies.
- (b) Every known row of σ' is a known row of σ , hence of τ . Every axis of τ is an axis of σ , hence of σ' . So τ completes σ' .
- (c) σ and σ' both lie below $\sigma \sqcup \sigma'$, so this is (b). $\square$

Theorem 8.8 (Soundness for programs). *For every program p , every $U \in \llbracket p \rrbracket$, and every σ with $\text{Sound}(U_0, \sigma)$,*

$$\text{Sound}(UU_0, \llbracket p \rrbracket^\sharp(\sigma)).$$

In particular $\llbracket p \rrbracket \subseteq \gamma(\llbracket p \rrbracket^\sharp(\sigma_0))$: every unitary the program can implement, over every choice of branches and every number of loop iterations, lies in the concretization of its abstract run.

Proof. By induction on the derivation of $U \in \llbracket p \rrbracket$.

- *Gates, sequencing, branches:* Lemma 8.7(a) and (c).
- *Loops.* Let I be the loop's result. Two properties suffice.
 - $\sigma \sqsubseteq I$, since each I_{k+1} is a join with I_k . This covers zero iterations, by Lemma 8.7(b).
 - $\llbracket p \rrbracket^\sharp(I) \sqsubseteq I$, either by the check or because $I = \top$. So if $\text{Sound}(UU_0, I)$ and $V \in \llbracket p \rrbracket$, the induction hypothesis for the body gives $\text{Sound}(VUU_0, \llbracket p \rrbracket^\sharp(I))$, and Lemma 8.7(b) brings it back to I . This covers one more iteration.

The last claim follows from $\text{Sound}(I, \sigma_0)$ and Lemma 8.6. $\square$

8.4 Examples

The next three examples are the looping programs *Loop-simple*, *Loop-h* and *Loop-swap* of Amy and Lunderville. We write a, b for qubits 1, 2, and $?$ for an unknown row.

Example 8.9 (A branch: **if** $\star$ **then** $\top_a$ **else** skip). Neither branch changes the rows, so the join keeps them all. The axes are (Z_a) from the first branch, joined with $()$ from the second, giving (Z_a) . The fact (Z_a, Z_a) survives, but facts about X_a do not: the first branch may have rotated it.

Example 8.10 ($\top_a$; **while** $\star$ **do** $CX_{a,b}$; $\top_a^\dagger$). After $\top_a$, the state has the initial rows and the axis Z_a . The body sets $x_a \mapsto x_a x_b = X_a X_b$ and $z_b \mapsto z_a z_b = Z_a Z_b$.

	x_a	x_b	z_a	z_b
I_0	X_a	X_b	Z_a	Z_b
$\llbracket \text{body} \rrbracket^\#(I_0)$	$X_a X_b$	X_b	Z_a	$Z_a Z_b$
$I_1 = I_0 \sqcup \llbracket \text{body} \rrbracket^\#(I_0)$	$?$	X_b	Z_a	$?$
$\llbracket \text{body} \rrbracket^\#(I_1)$	$?$	X_b	Z_a	$?$

The check $\llbracket \text{body} \rrbracket^\#(I_1) \sqsubseteq I_1$ passes: the unknown rows stay unknown, since $? \cdot X_b = ?$, and the body adds no axes. After the loop, $\top_a^\dagger$ needs $z_a = Z_a$, which is known. So the final axes are (Z_a, Z_a) and $\beta(Z_a) = Z_a$.

The fact (Z_a, Z_a) holds for *every* number of iterations. It is what lets an optimizer merge the $\top$ with the $\top^\dagger$: the program's T -count goes from 2 to 0, as in Amy and Lunderville.

Example 8.11 ($\top_b$; **while** $\star$ **do** H_a ; $\top_b^\dagger$). The body swaps x_a and z_a , so the join forgets both. The rows of b are never touched, so they survive, and $\top_b^\dagger$ records the known axis Z_b . The fact (Z_b, Z_b) holds for every execution.

Example 8.12 (Where the join is too coarse: a swap loop). Take $CX_{a,b}$; $\top_b$; $CX_{a,b}$; **while** $\star$ **do** $\text{SWAP}_{a,b}$; $CX_{a,b}$; $\top_b^\dagger$; $CX_{a,b}$, with SWAP written as three CNOTs.

- *The loop.* The swap exchanges the rows of a and b , so the join forgets all four of them.
- *After the loop.* The CX sets $z_b = z_a z_b = ?$, and $\top_b^\dagger$ then has an unknown axis, so the result is $\top$.

Yet the two branches of the loop (zero and one iteration) agree on $\beta(Z_a Z_b)$: $z_a z_b$ and $z_b z_a$ are equal. A join that keeps every Pauli the two branches agree on, not just every row, would keep the fact $(Z_a Z_b, Z_a Z_b)$. That is the loop invariant $x' \oplus y' = x \oplus y$ of Amy and Lunderville. The per-row join trades this precision for simplicity. Section 8.5 recovers it with a disjunctive domain.

8.5 A bounded disjunctive domain

Instead of merging branches into one partial state, keep them apart.

Definition 8.13 (Disjunctive states). A *disjunctive state* is either $\top$ or a finite list $S = (\sigma_1, \dots, \sigma_m)$ of partial states. It is read as “the program is described by one of the σ_i ”.

- *Concretization:* $\gamma(S) = \bigcup_i \gamma(\sigma_i)$. A fact therefore holds for the program when it is a fact of every σ_i .
- *Transformers:* applied to each σ_i . If one reaches $\top$, so does the whole state.
- *Normalization:* disjuncts with the same rows are merged, keeping the rows and taking the union of the axes. If more than K disjuncts remain, for a fixed bound K , they are collapsed into one with the per-row join.
- *Join:* $S \sqcup S'$ is the normalization of the concatenation of S and S' .
- *Order:* $S \sqsubseteq S'$ when every disjunct of S lies below some disjunct of S' . Loops are iterated exactly as before, with this join and order.

Merging disjuncts with equal rows is what makes loops with rotations in the body stabilize. Each iteration adds axes, but not new rows.

Theorem 8.14 (Soundness). *Write $\text{Sound}(U, S)$ when some disjunct σ_i of S has $\text{Sound}(U, \sigma_i)$. Theorem 8.8 holds verbatim for disjunctive states, and in particular $\llbracket p \rrbracket \subseteq \gamma(\llbracket p \rrbracket^\#(\sigma_0))$.*

Proof. Transformers act disjunct by disjunct, so Lemma 8.7(a) applies to the sound disjunct. For the join and the order, every disjunct of S lies below some disjunct of the normalization of $S \cdot S'$:

- *Merging* equal rows only adds axes.
- *Collapsing* is a per-row join, which lies above each of its arguments.

So Lemma 8.7(b) moves soundness from a disjunct to the disjunct above it. The loop argument is unchanged. $\square$

Example 8.15 (The swap loop, again). For **while** $\star$ **do** $\text{SWAP}_{a,b}$, starting from the initial rows R_0 :

- *The disjuncts.* The body gives the swapped rows R_1 , with $x_a = X_b$, $x_b = X_a$, $z_a = Z_b$ and $z_b = Z_a$. Joining gives the disjuncts (R_0, R_1) .
- *Closure.* The body maps R_1 back to R_0 , so (R_0, R_1) is closed and the check passes at once.
- *The fact.* In R_0 , $\beta(Z_a Z_b) = Z_a Z_b$. In R_1 , $\beta(Z_a Z_b) = z_a z_b = Z_b Z_a = Z_a Z_b$. Neither disjunct has axes, so $(Z_a Z_b, Z_a Z_b)$ is a fact of the loop.

In the full *Loop-swap* program the T and the $\mathsf{T}^\dagger$ both act on the parity $Z_a Z_b$, so they merge and cancel. The T -count goes from 2 to 0, as in Amy and Lunderville.

For **while** $\star$ **do** $(\text{SWAP}_{a,b}; \text{SWAP}_{b,c})$ the disjuncts are the three rotations of the three qubits. Qubit b receives each value in turn, so Z_b is not preserved, correctly.

Remark 8.16 (Which programs benefit). A disjunct is a Clifford the program may have applied. The domain is precise when a loop body's Clifford part generates a small group, at most K elements: swaps, CNOTs and Hadamards, which have order 2, and cycles of qubits. A body whose Clifford part has large order, or a program with many independent branches, exceeds the bound and falls back to the per-row join.

9 The optimizer: PhaseFoldPauli

This section describes the Rust pass `PhaseFoldPauli` (`src/pauli_fold_rand.rs`), which uses the tableau abstraction to reduce T -count. It adds two things to the abstraction.

- *Angles.* Each recorded axis also carries its rotation's angle and sign.
- *The fold rule.* Two rotations with the same axis merge when every axis recorded between them commutes with it.

The pass is a single forward scan over the circuit. It rewrites gates in place and never reorders them. We present the algorithm without the engineering that makes the implementation fast. Section 10 describes it; it changes the cost of the scan, not its verdicts.

9.1 What the pass keeps

The frame. The frame holds the rows x_q and z_q of the tableau, and Clifford gates update them as in Table 2. Each row is a Pauli string with a sign, $\pm X^u Z^w$ where $u, w \in \mathbb{F}_2^n$.

Events. The *history* is the list of axes recorded so far, in order. Each entry is an *event*

$$e = (\text{gate index}, A_e, s_e, \theta_e, \text{live}).$$

- A_e is the *axis*: the row z_q at the time of the rotation, without its sign.
- $s_e = \pm 1$ is the row's sign.
- θ_e is the angle.
- *live* says the rotation is still in the circuit at this position. An event becomes *dead* when its rotation is merged into a later one.

Some events have no gate and no angle. These *blockers* never merge, but a fold must commute with them to move across them.

- A CCX or CCZ contributes the seven axes of its decomposition into $\pi/8$ rotations.
- A measurement contributes z_q .
- A reset contributes z_q and x_q .

9.2 The scan

Algorithm 1 PhaseFoldPauli (one pass).

```

1: frame  $\leftarrow$  the identity tableau; history  $\leftarrow$  empty
2: for each gate  $g_j$ , in order do
3:   if  $g_j$  is Clifford then
4:     update the frame by Table 2
5:   else if  $g_j$  is a rotation by  $\theta_j$  on qubit  $q$  then
6:      $A \leftarrow z_q$  without its sign;  $s_j \leftarrow$  the sign of  $z_q$ 
7:      $i \leftarrow$  the newest live event with axis  $A$ 
8:     if  $i$  exists and  $A$  commutes with every live event after  $i$  then
9:        $\theta \leftarrow s_i s_j \theta_i + \theta_j$   $\triangleright$  merge
10:      delete gate  $i$ ; mark event  $i$  dead; give gate  $j$  the angle  $\theta$ 
11:      if  $\theta$  is a multiple of  $\pi/2$  then
12:        emit S,  $S^\dagger$ , Z or nothing, and apply it to the frame
13:        continue  $\triangleright$  a Clifford: no event
14:      end if
15:       $\theta_j \leftarrow \theta$ 
16:    end if
17:    append the event  $(j, A, s_j, \theta_j, \text{live})$ 
18:  else if  $g_j$  is CCX, CCZ, a measurement or a reset then
19:    append its blocker events
20:  end if
21: end for
22: return the circuit, with deleted gates removed and merged angles emitted

```

9.3 Correctness: the fold theorem

The merge on lines 9–12 of Algorithm 1 is justified by one fact of the abstract state of the segment *between* the two rotations. We first state the theorem for an arbitrary segment. We then show that the check on line 8 establishes its hypothesis.

Rotations. For a Pauli string P with $P^2 = I$ and an angle θ , let

$$R_P(\theta) = e^{-i\theta P/2} = \cos \frac{\theta}{2} I - i \sin \frac{\theta}{2} P.$$

Up to a global phase, $\mathsf{T} = R_Z(\pi/4)$, $\mathsf{T}^\dagger = R_Z(-\pi/4)$ and $\mathsf{S} = R_Z(\pi/2)$. Angles add along a fixed axis: $R_P(\theta)R_P(\theta') = R_P(\theta + \theta')$. We write circuits left to right, so $\llbracket cc' \rrbracket = \llbracket c' \rrbracket \llbracket c \rrbracket$.

Theorem 9.1 (Fold). *Let m be a circuit and $\sigma = \llbracket m \rrbracket^\sharp$. Suppose that, for qubits q, q' and a sign $s = \pm 1$:*

- (a) $\beta_\sigma(Z_{q'}) = s Z_q$;
- (b) Z_q commutes with every axis of σ .

Then $\llbracket m \rrbracket R_{Z_q}(\theta) = R_{Z_{q'}}(s\theta) \llbracket m \rrbracket$ for every θ . Hence, for all circuits ℓ, k and all angles θ, θ' ,

$$\llbracket \ell R_{Z_q}(\theta) m R_{Z_{q'}}(\theta') k \rrbracket = \llbracket \ell m R_{Z_{q'}}(s\theta + \theta') k \rrbracket.$$

In words, (a) and (b) together say that $(sZ_q, Z_{q'})$ is a fact of σ . The segment maps the axis of the first rotation onto the qubit of the second one, and no rotation inside the segment disturbs it. The theorem uses nothing else about m .

Proof. Let $M = \llbracket m \rrbracket$.

1. *The fact.* Take $P = Z_{q'}$ in Theorem 5.1. Its pull-back sZ_q commutes with every axis by (b), since a sign does not affect commutation. So $M(sZ_q)M^\dagger = Z_{q'}$, that is, $MZ_qM^\dagger = sZ_{q'}$.
2. *Slide.* Conjugating the rotation term by term,

$$MR_{Z_q}(\theta)M^\dagger = \cos \frac{\theta}{2} I - i \sin \frac{\theta}{2} sZ_{q'} = R_{Z_{q'}}(s\theta).$$

Multiplying on the right by M gives the first claim.

3. *Merge.* Then

$$\llbracket \ell R_{Z_q}(\theta) m R_{Z_{q'}}(\theta') k \rrbracket = K R_{Z_{q'}}(\theta') M R_{Z_q}(\theta) L = K R_{Z_{q'}}(\theta') R_{Z_{q'}}(s\theta) ML,$$

and the two rotations about $Z_{q'}$ combine into one by angle $s\theta + \theta'$. □

Step 1 uses only $M \in \gamma(\sigma)$, not the circuit m itself. So the same argument applies to every unitary a program can implement.

Corollary 9.2 (Folding across branches and loops). *Let m be a program in the sense of Section 8. Let σ be its abstract result from the identity tableau, in the per-row or the disjunctive domain. Suppose σ satisfies (a) and (b), and in the disjunctive domain assume this of every disjunct. Then $UR_{Z_q}(\theta) = R_{Z_{q'}}(s\theta)U$ for every $U \in \llbracket m \rrbracket$. So $\ell R_{Z_q}(\theta) m R_{Z_{q'}}(\theta') k$ and $\ell m R_{Z_{q'}}(s\theta + \theta') k$ implement the same set of unitaries.*

Proof. By Theorem 8.8, or Theorem 8.14, every $U \in \llbracket m \rrbracket$ lies in $\gamma(\sigma)$. Then argue as in the proof of Theorem 9.1, path by path. □

The scan checks the hypothesis. Algorithm 1 never computes $\llbracket m \rrbracket^\sharp$ for a segment. It reads the hypothesis off the global frame and the history instead.

Lemma 9.3 (The scan's check). *Consider line 8 at gate j on qubit q' , with candidate event i on qubit q . Let m be the current circuit strictly between i and j , after all earlier rewrites. If A commutes with every live event between i and j , then $\llbracket m \rrbracket^\sharp$ satisfies (a) and (b) with $s = s_i s_j$.*

Proof. The frame follows the current circuit. Before each gate, the frame is the Clifford part of the current circuit up to that gate. Deleted rotations never changed the frame, and a Clifford emitted by a merge is applied to the frame at its own position.

Let C_i and C_j be the frames before gates i and j , and let C_m be the Clifford part of m . Then $C_j = C_m C_i$. By Lemma 2.4,

$$C_i^\dagger Z_q C_i = s_i A \quad \text{and} \quad C_j^\dagger Z_{q'} C_j = s_j A.$$

(a). The pull-back through m is

$$\beta_{\llbracket m \rrbracket^\#}(Z_{q'}) = C_m^\dagger Z_{q'} C_m = C_i (C_j^\dagger Z_{q'} C_j) C_i^\dagger = s_j C_i A C_i^\dagger = s_i s_j Z_q.$$

(b). The rotations of m are exactly the live events between i and j . A dead event's gate has been deleted, and its angle now sits at a live event or an emitted Clifford inside the interval. Take a rotation of m at gate k on qubit r , with global axis A_k , so $C_k^\dagger Z_r C_k = s_k A_k$. The Clifford part of m before it is $C_k C_i^{-1}$, so its axis in m 's own frame is

$$(C_k C_i^{-1})^\dagger Z_r (C_k C_i^{-1}) = s_k C_i A_k C_i^\dagger.$$

Conjugation by C_i preserves commutation, and $Z_q = s_i C_i A C_i^\dagger$. So Z_q commutes with this axis exactly when A commutes with A_k , which is what line 8 checks. $\square$

Theorem 9.1 with Lemma 9.3 is exactly the rewrite on lines 9–12. The merged angle is $s_i s_j \theta_i + \theta_j$, gate i is deleted, and the merged rotation sits at j .

Why blockers are safe. A blocker's operator is a combination of Pauli strings built from its axes: the Toffoli's $\pi/8$ rotations, the projectors $(I \pm z_q)/2$ of a measurement, or the Kraus maps of a reset, which are built from z_q and x_q . So it commutes with every rotation whose axis commutes with its axes.

Why the newest candidate suffices. Suppose an older live event $i' < i$ also has axis A . Its interval (i', j) contains (i, j) , so anything that blocks i also blocks i' .

Cost.

- A Clifford gate changes at most two rows, in $O(n)$ time.
- A rotation compares its axis with earlier events and checks commutation with the events after the candidate, in $O(rn)$ time for r events.

So a pass on m gates with r rotations takes $O(mn + r^2 n)$ time.

9.4 A worked example

Figure 1 shows a ten-gate circuit on two qubits. Table 3 traces the scan. Rows are written in input coordinates, and e_k is the k -th event.

The output is the circuit with gate 1 deleted and gate 5 replaced by **S**. It has three T gates instead of five; this is exactly what `tzap -passes PhaseFoldPauli` produces. Without gates 6–8 the T at gate 3 and the $T^\dagger$ at gate 10 would merge and cancel as well.

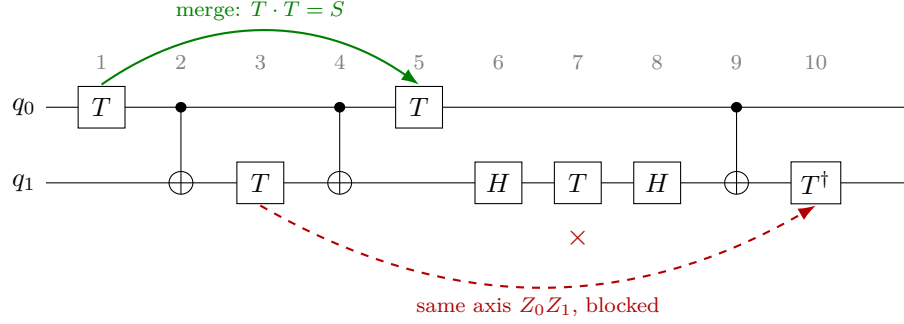


Figure 1: A circuit for Algorithm 1. The T at gate 1 slides past the CX– T –CX block, whose rotation acts on Z_0Z_1 and commutes with Z_0 , and merges into gate 5 as an S . The T at gate 3 and the $T^\dagger$ at gate 10 share the axis Z_0Z_1 . The T at gate 7 has axis X_1 , which anticommutes with Z_0Z_1 , so they cannot meet.

9.5 Relation to the abstraction

- *The frame.* The frame is the tableau of Section 2, with the transformer of Table 2.
- *The history.* The live events are its axes, each annotated with an angle, with blockers added for Toffolis, measurements and resets.
- *The fold rule.* “Same axis, and every axis recorded between them commutes with it” is the fact $(sZ_q, Z_{q'})$ of the segment’s abstract state (Lemma 9.3). By Theorem 9.1, that single fact licenses the merge.
- *Order.* The history is ordered only so that one global list serves every segment. The segment’s own axes are a contiguous slice of it, conjugated by C_i .

Theorem 9.1 is proved in Lean (`FoldRule.lean`), with phase-exact rotations $\text{diag}(1, e^{i\pi\theta})$ in place of $R_P(\theta)$. For the sign $-$, the two versions differ by the global phase $e^{i\pi\theta}$. Corollary 9.2 and Lemma 9.3 are argued on paper only.

10 Making the scan fast

Algorithm 1 as written costs $O(rn)$ per rotation: it compares a new axis with every earlier event, and it tests commutation against every event since the candidate. On circuits with 10^7 gates this is far too slow. This section describes how the Rust pass gets its speed, in the order the pieces matter. Every optimization here either answers the same question faster or refuses a fold. None of them can authorize a rewrite that Algorithm 1 would reject.

10.1 Finding the candidate: fingerprints

Fingerprints. Each generator X_q, Z_q gets a random 128-bit label $\lambda(X_q), \lambda(Z_q)$. The *fingerprint* of a Pauli string, ignoring its sign, is the XOR of the labels of the generators it contains, with Y_q counting as both X_q and Z_q :

$$f(X^u Z^w) = \bigoplus_{u_q=1} \lambda(X_q) \oplus \bigoplus_{w_q=1} \lambda(Z_q).$$

Multiplying Pauli strings XORs their bit vectors, so $f(PQ) = f(P) \oplus f(Q)$. Every row update of Table 2 is a swap, a sign change, or a product of two rows. So a second, *sketch* frame can hold

	gate	x_0, z_0	x_1, z_1	axis	action
	start	X_0, Z_0	X_1, Z_1		
1	T_0	X_0, Z_0	X_1, Z_1	$+Z_0$	no earlier axis Z_0 : record e_1
2	$CX_{0,1}$	X_0X_1, Z_0	X_1, Z_0Z_1		$x_0 \leftarrow x_0x_1, z_1 \leftarrow z_0z_1$
3	T_1			$+Z_0Z_1$	no earlier axis Z_0Z_1 : record e_2
4	$CX_{0,1}$	X_0, Z_0	X_1, Z_1		the rows return
5	T_0			$+Z_0$	candidate e_1 . After e_1 : $e_2 = Z_0Z_1$ commutes with Z_0 . Merge : $\frac{\pi}{4} + \frac{\pi}{4} = \frac{\pi}{2}$. Delete gate 1, emit S at 5, kill e_1 , and apply S to the frame.
	(S)	$-Y_0, Z_0$			$x_0 \leftarrow -ix_0z_0 = -iX_0Z_0 = -Y_0$
6	H_1		Z_1, X_1		swap x_1, z_1
7	T_1			$+X_1$	no earlier axis X_1 : record e_3
8	H_1		X_1, Z_1		swap back
9	$CX_{0,1}$	$-Y_0X_1, Z_0$	X_1, Z_0Z_1		
10	$T_1^\dagger$			$+Z_0Z_1$	candidate e_2 . After e_2 : $e_3 = X_1$ anti-commutes with Z_0Z_1 . Blocked : record e_4 .

Table 3: The scan of Figure 1. Empty cells are unchanged.

the fingerprints of the rows and update them with the same pattern, at $O(1)$ per gate instead of $O(n/64)$.

The candidate index. A hash map sends each fingerprint to its newest event, and each event links to the previous event with the same fingerprint. The candidate search on line 7 is then one lookup, followed by exact axis comparisons along that chain until one matches. Two cases remain:

- *Collisions.* Different axes collide with probability 2^{-128} . A collision only costs a wasted comparison, because the exact axes decide.
- *Dead links.* Links to dead events are skipped and then compressed, so a long run of folds on one axis does not make later lookups walk it again.

The randomness therefore affects only speed. The output is deterministic.

The pre-check. Before building anything, the pass runs the sketch frame alone and records the fingerprint of every rotation's axis. If no fingerprint repeats, no two rotations share an axis, and the circuit is returned unchanged without ever building the exact frame.

10.2 Checking commutation: support lists

Support lists. Two Pauli strings can anticommute only on a qubit where both act. For each qubit block b (the qubits are split into at most 64 blocks), the pass keeps the list of events whose axis acts on b , in increasing order. To check line 8, it looks at the lists of the blocks where A acts. In each one, it binary-searches for the first event after the candidate and tests the events from there.

Test each event once. An event that shares several blocks with A appears in several of these lists. Each event records its *signature*, the 64-bit mask of blocks it acts on. It is tested only in the list of the lowest block it shares with A , that is, when $\text{ctz}(\text{sig}_e \wedge \text{sig}_A)$ is the current block.

Liveness without indirection. The scan needs each event’s liveness and axis. Liveness is kept in a separate bitset, one bit per event. Axes are stored contiguously, $2\lceil n/64 \rceil$ words each, so event e ’s axis starts at a fixed offset. The inner loop therefore reads two small arrays instead of following a pointer into a large event record.

10.3 The dominant cost: folded rotations

The problem. On the large cobble circuits, 98% of the T gates fold away. Their events die, but their ids stayed in the support lists. We measured the lists on `ols-ridge-d24` (20M gates, 28 qubits):

- the commutation checks visited 6.2×10^9 list entries;
- 98% of those entries were dead;
- the check took 88% of the pass.

Most of the visits came from *blocked* attempts. A rotation whose axis last appeared tens of thousands of events earlier has a long window. The scan then walks thousands of dead entries before it meets the live event that blocks the fold.

Compaction. Each list keeps a count of its dead entries. When a rotation dies, the count goes up in each of its lists. A list is compacted, keeping only live ids and preserving their order, once a quarter of it is dead. Each compaction is linear in the list, and at least a quarter of its entries died since the previous one, so the cost is $O(1)$ amortized per death. On `ols-ridge-d24` this cut the visits from 6.2×10^9 to 3.6×10^8 , and the output did not change.

Scan order does not help. The verdict, whether some live event in the window anticommutes with A , does not depend on the order of the scan. We tried scanning newest first, and the result depended on the circuit:

- on some circuits it was $2\times$ faster;
- on the `ols-ridge` family it was $3.5\times$ slower, because there the blockers sit near the old end of the window.

Alternating between the two ends gave middling results everywhere. The scan stays oldest-first.

10.4 Bit-sliced lists

The remaining cost is testing one event at a time. For circuits with at most 32 qubits, an axis is one X word and one Z word. Two axes $P = (x, z)$ and $Q = (x', z')$ anticommute exactly when

$$\omega(P, Q) = \text{popcount}((x \wedge z') \oplus (z \wedge x')) \bmod 2 = 1.$$

Its terms can be regrouped by qubit, so the test runs on 64 events at once.

The layout. Each qubit’s list is stored in blocks of 64 entries. Entry i of a block owns bit i of every mask in that block. A block holds, for each qubit p :

- X_p , the mask of entries whose axis has X or Y on p ;

- Z_p , the mask of entries with Z or Y on p ;
and one mask L of live entries.

The test. Anticommutation with $A = (a_x, a_z)$ is the parity, over the qubits p of A 's support, of “ A has X on p and the entry has Z on p ” plus “ A has Z on p and the entry has X on p ”. For all 64 entries at once, that is

$$\text{hit} = \left(\bigoplus_{p: a_x(p)=1} Z_p \oplus \bigoplus_{p: a_z(p)=1} X_p \right) \wedge L.$$

Any set bit is a live anticommuting event, so the fold is blocked. A block costs $\text{wt}(A)$ word XORs instead of 64 separate tests. The per-entry storage is small: $2n$ bits of masks plus a 32-bit id, about 11 bytes for $n = 28$.

Example 10.1 (One block). Take two qubits and a block with four entries, $e_0 = Z_0Z_1$, $e_1 = X_1$, $e_2 = X_0X_1$ and $e_3 = Y_0$, with e_1 dead. Writing masks as $e_3e_2e_1e_0$:

$$X_0 = 1100, \quad Z_0 = 1001, \quad X_1 = 0110, \quad Z_1 = 0001, \quad L = 1101.$$

- For $A = Z_1$, only $a_z(1) = 1$, so $\text{hit} = X_1 \wedge L = 0100$. That is $e_2 = X_0X_1$, the only live entry that anticommutes with Z_1 . The entry $e_1 = X_1$ also anticommutes, but it is dead.
- For $A = X_0Z_1$, $Z_0 \oplus X_1 = 1111$: every entry anticommutes with X_0Z_1 , and $\text{hit} = 1101$.

Updates.

- *A new event* with support w goes into w lists and sets up to w mask bits in each, so recording it costs $O(w^2)$ rather than $O(w)$.
- *A death* clears one bit of L in each of the event's lists. Its position there is found by a galloping search from the end, because rotations usually fold soon after they are recorded.
- *Compaction*. A list is rebuilt from its live entries once half of it is dead.

When to use it. Scans get cheaper and insertions get more expensive. The trade pays off when fold attempts scan long windows, as on the cobble circuits, and loses when axes are dense and few rotations fold, as on QFT. On 50-qubit QFT, bit-slicing made the pass $2.2\times$ slower. The pass therefore uses bit-sliced lists up to 32 qubits and the scalar lists above that. A test checks that the two kinds reach the same verdicts: with an unbounded budget, they produce identical rewrites on 600 random circuits with measurements, CCX and CCZ.

10.5 The budget

Each fold attempt gets a budget of 2^{16} steps. The candidate search and the commutation check share it.

- *Candidate search*: one step per live chain entry compared.
- *Scalar lists*: one step per list entry visited.
- *Bit-sliced lists*: one step per live entry in each block scanned.

An attempt that runs out counts as blocked. The rotation is then recorded as a new event, and the scan goes on. This bounds the pass by $O(m + r \cdot 2^{16})$. It is sound, because it only refuses folds.

Once the scan was fast, the budget stopped mattering. Across 63 benchmark circuits:

budget per attempt	pass time	rotations left	lost folds vs. 2^{20}
2^8	6.9 s	13,220,606	10,823,123
2^{10}	5.8 s	2,402,329	4,846
2^{12}	5.9 s	2,398,399	916
2^{16}	6.1 s	2,397,821	338
2^{20}	6.2 s	2,397,483	0

Below about 2^{10} folds collapse, and the pass even gets slower: unfolded rotations stay live and lengthen every list. Above it, the time moves by less than 7%.

10.6 Results

These timings are for the pass alone: one thread, on the whole circuit, timed in process on the same parsed input. **PhaseFoldRand** is shown for scale.

circuit	gates	original	compaction	all	PhaseFoldRand
ols-ridge-d24	19.8M	3.73 s	3.01 s	2.42 s	0.56 s
hamiltonian-n18	10.6M	4.97 s	2.27 s	0.97 s	0.24 s
chebyshev-n36	10.4M	2.32 s	1.50 s	0.99 s	0.25 s
spectral-n29	10.2M	11.69 s	6.54 s	1.14 s	0.26 s
all 63 circuits		23.6 s	13.8 s	6.2 s	1.5 s

Over all 63 circuits the pass is $3.8\times$ faster than it was. It still costs about four times **PhaseFoldRand**, but it leaves 2.4M rotations to **PhaseFoldRand**’s 19.4M. On the 97-circuit corpus, the CLI output is byte-identical on 93 circuits. The other 4 have a few more folds, from attempts that used to exhaust the budget on dead entries.

The CLI splits a circuit into chunks and runs them in parallel. Each chunk then has a short history, so there the gain is larger. On **ols-ridge-d24**, compaction alone took the pass itself from about 5 s to about 0.1 s, against a trivial pass on the same file. The price is that chunking gives up the folds that would cross a chunk boundary.

What remains is spread out: the commutation check, deaths, the candidate hash lookup, and the frame updates each take 10–30% of the pass. A further step would choose bit-sliced lists by average axis density rather than by qubit count, to recover QFT’s 20- and 30-qubit cases, which are now $0.75\text{--}0.83\times$ their old speed.

11 Comparison with Amy and Lunderville’s relational analyses

Amy and Lunderville (POPL 2025) recast phase folding as a *relational analysis*. A program is abstracted by the classical transitions it can make: the pairs of basis states (x, x') with $\langle x'|U|x\rangle \neq 0$. They give two domains for these transitions:

- PF_{Aff} , affine relations over $\mathbb{F}_2$;
- PF_{Pol} , polynomial relations, as ideals with Gröbner bases.

They also give a way to strengthen either one by rewriting the program’s path sum (*Strengthen*).

Scope of the Lean comparisons. Lean’s **StateFold** and **Strengthen** are *restricted* models of these analyses. They keep only facts $Z_q \mapsto \pm Z_{q'}$, not general transition relations. Their witnessed constraints are taken on the final path sum only, not accumulated as in Algorithm 2,

and the degree bound d limits only HH substitutions. Separations in which the path-sum analysis proves more transfer to the full domain. `Alpha.lean` also defines the paper’s path-sum abstraction $\alpha = \exists Y. \langle X' \oplus f(X, Y) \rangle$ (their Definition 17) and Algorithm 1’s rewriting. It proves soundness and monotonicity (their Propositions 18 and 21), and it proves both separations against α itself (`alpha_pauli_incomparable`). Only Strengthen’s accumulated ideal is still modeled in restricted form.

Domain versus analysis. A separation can come from what a domain can *express*, or from what an analysis manages to *compute*.

- *Expressiveness.* Every affine relation is a Z-type Pauli fact, so the tableau’s facts cover affine relations and add X- and Y-type facts. Polynomial relations and Pauli facts are incomparable: $x' = x \oplus ab$ is not a Pauli fact, and $X \mapsto X$ is not a relation on the computational basis.
- *What is computed.* The relational analyses compute their abstraction by rewriting with a fixed rule set, which can fall short of the best abstraction $\alpha(U)$.

This section compares those analyses with the tableau domain. It states what each one records and where each one is more precise, with an example for every separation.

11.1 What each analysis records

	PF _{Aff}	PF _{Pol} (+ Strengthen)	tableau (this note)
state	affine relations among the outputs x' , inputs x and path variables y	polynomial relations among x', x, y	rows x_q, z_q (the Clifford part) and axes
facts	$x'_S = x_T \oplus c$	$f(x', x) = 0$	$U \beta(P) U^\dagger = P$ for all P with $\beta(P)$ commuting with the axes
H	fresh variable y (x'_q is forgotten)	fresh variable, removed later by path-sum rewriting	exact: swap the rows
T, R_Z	a phase term on a linear function; the relation is unchanged	the same	records the axis z_q
CCX	target forgotten	exact: $x'_t = x_t \oplus x_a x_b$	seven axes
measure	$x'_q = x_q$	$x'_q = x_q$	the axis z_q
reset	$x'_q = 0$ (a state fact)	$x'_q = 0$	the axes z_q, x_q (no state fact)
join	affine hull	product of ideals	per-row, or disjunctive (Section 8.5)
merge rule	same linear function modulo the relation (their Prop. 9)	the same, modulo the ideal	Theorem 9.1

Table 4: The three analyses side by side.

The bridge: relations are Z -facts. An affine relation says the parity of the outputs on a set S equals the parity of the inputs on a set T , plus a constant c . This is exactly the Pauli fact

$$U Z_T U^\dagger = (-1)^c Z_S,$$

where $Z_T = \prod_{q \in T} Z_q$. Lean proves both directions: a relation gives the fact (`rel_pauliFact`), and a unitary with the fact satisfies the relation on its whole support (`support_of_zfact`).

So the affine domain speaks only about Z -type facts, meaning facts about the computational basis. The tableau speaks about every Pauli, and it treats H exactly. Their merge rule, Proposition 9, requires $x'_j = x_i$ on every transition of the segment. That is Theorem 9.1 with the segment fact (Z_i, Z_j) . For straight-line circuits, this gives:

Theorem 11.1 (Tableau refines affine, straight-line; Lean). *For every Clifford+ T circuit c , $\gamma(\llbracket c \rrbracket^\sharp) \subseteq \gamma_{\text{Aff}}(c)$, and the inclusion is strict on HH .*

Here γ_{Aff} is the affine analysis without path-sum rewriting, with H as a fresh variable. In Lean these are `affGamma` and `tableau_refines_aff_strict`. Path-sum rewriting and non-linear relations add facts that the tableau cannot express, and there the domains become incomparable (Section 11.2).

11.2 Examples

Throughout, “folds” means the analysis proves the fact that Theorem 9.1 (or their Proposition 9) needs.

Tool check. The straight-line examples were run through both tools.

- *tzap*. `tzap -passes PhaseFoldPauli`, with each output checked for equivalence by simulation.
- *Feynman*. `feynopt -verify`, at commit 4c4c488, through its `.qc` frontend. Every output verified. The QASM2 frontend with `-purecircuit` gives the same counts, and its outputs are equivalent by simulation.

In the table, `sf  $d$` is `-statefold  $d$` , and $d = 0$ means unbounded.

example	T in	tzap	-phasefold	sf 1 (affine)	sf 2	sf 0
11.2 H frame	2	0	2	0	0	0
11.3 Clifford segment	2	0	2	2	2	2
11.4 relative Toffoli	11	11	11	11	9	9

Feynman’s `-apf` and `-ppf` pipelines also fold Example 11.3 to 0. They include Feynman’s own Pauli-folding pass (`Paulifold 1`), which is not part of the relational analyses. The loop and reset examples, and the tableau’s entries for them, are hand analyses: `tzap` reads no loops, and Feynman’s verifier covers only `.qc` circuits. The QASM2 frontend gives the same counts with `-purecircuit`. Without that flag it assumes every qubit starts in $|0\rangle$, a different semantics under which, for example, a leading T is removed.

Example 11.2 (An H frame). Take $H_a T_a H_a CX_{b,a} H_a T_a H_a$.

- *Tableau*. Both T gates have axis X_a , since $CX_{b,a}$ does not change x_a . So they merge into an S , and 2 T become 0.

- *Affine*. The four H gates introduce $y_1, \dots, y_4$. The T gates act on y_1 and y_3 , and no relation links them, so nothing folds.
- *Path sums*. The phase contains $y_2(y_1 \oplus y_3)$. Summing out y_2 (the (H) rule) witnesses $y_1 = y_3$, so Strengthen folds too.

The tableau gets this fold directly from the row update for H , with no rewriting.

Example 11.3 (A Clifford segment). Take $T_1 CX_{0,1} H_0 H_1 CX_{1,0} H_1 T_1^\dagger$.

- *Tableau*. The segment between the rotations is Clifford and maps Z_1 to Z_1 . So the T and $T^\dagger$ cancel, and 2 T become 0.
- *Path sums*. Both output wires mention every path variable, so neither rewrite rule applies, and there is no witness. The wire map is onto, so α is $\top$, and nothing folds (`alpha_cexMid`). This is a limit of the rules, not of the domain. The exact $\alpha(U)$ contains $x'_1 = x_1$. Renaming the q_0 wire $u = y_0 \oplus y_1$ turns the phase into $x_0 u + y_1(x_1 \oplus y_2)$. An (H) step on y_1 then gives $y_2 = x_1$, and Feynman's `-cxcz` preprocessing finds exactly this.
- *Affine*. The H gates forget the wires.

So the tableau is not below Algorithm 1 as the circuit is written. This is a separation of analyses, not of domains.

Example 11.4 (A relative-phase Toffoli). Take the 21-gate circuit $T_0 G T_0^\dagger G T_0$ (in Lean, `DegreeTwo.lean`), where G is a relative-phase Toffoli with target 0.

- *Path sums, degree 2*. The analysis proves that the outer axis Z_0 returns unchanged (`midE_ssubset`), because G computes and then uncomputes $x_0 \oplus x_1 x_2$.
- *Tableau*. It records axes inside G that anticommute with Z_0 , so the outer T gates are blocked, and all 11 remain.

The missing ingredient is a *non-linear* classical function. The tableau sees G only through its axes.

Example 11.5 (State knowledge: *Reset-simple*). Take T_a ; `reset a`; T_a .

- *Affine*. The reset sets $x'_a = 0$, so the second T has phase condition 0 and is deleted, leaving 1 T .
- *Tableau*. The reset only adds the axes z_a and x_a , so both T remain.

Recovering this needs stabilizer facts $P \cdot K = K$ about the current state, an extension of the domain not developed here. *Loop-nested* fails for the same reason: its T_a acts on a qubit that was reset before the loop.

Example 11.6 (A loop in an H frame). Take T_a ; **while** $\star$ **do** ($H_a CX_{b,a} H_a$); $T_a^\dagger$. The body is $CZ_{a,b}$ written with H gates.

- *Tableau*. The body maps $z_a \mapsto Z_a$ and $z_b \mapsto Z_b$. The per-row join with the identity keeps both z rows and forgets the x rows. So the T and $T^\dagger$ cancel.
- *Affine*. The body's H gates forget x_a , so the loop summary has no relation on a , and nothing folds.
- *Path sums*. Summarizing the body by path-sum rewriting recovers $x'_a = x_a$, so PF_{Pol} with Strengthen folds too.

Example 11.7 (An S_4 loop: the affine hull beats both joins). Let the body of **while** $\star$ be a nondeterministic choice between $SWAP_{a,b}$ and the 4-cycle $(abcd)$. Around the loop, place a T and a $T^\dagger$ on the parity $x_a \oplus x_b \oplus x_c \oplus x_d$, each conjugated by a CNOT ladder.

- *Affine*. Every permutation preserves the total parity, so the affine hull of the reachable permutations keeps $x'_a \oplus x'_b \oplus x'_c \oplus x'_d = x_a \oplus x_b \oplus x_c \oplus x_d$, and the rotations fold.

- *Per-row join*. It forgets every row after one iteration, giving $\top$.
- *Disjunctive join*. It must keep all 24 permutations of S_4 . It exceeds $bound = 8$ and collapses to the per-row join, giving $\top$.

A join that keeps the *subgroup* of Paulis on which two states agree, $\{P : \beta_1(P) = \beta_2(P)\}$, would keep $Z_a Z_b Z_c Z_d$ here. It is computable by linear algebra over $\mathbb{F}_2$, and it would subsume the affine hull on Z -type facts. We have not built it.

11.3 Summary

example	plain affine	path sums / Pol	tableau, per-row	tableau, disjunctive
11.2 H frame	–	✓	✓	✓
11.3 Clifford segment	–	–	✓	✓
11.4 relative Toffoli	–	✓ ($d \geq 2$)	–	–
11.5 reset	✓	✓	–	–
11.6 loop in H frame	–	✓	✓	✓
Loop-swap (Ex. 8.12)	✓	✓	–	✓
11.7 S_4 loop	✓	✓	–	–

Table 5: Which analyses fold each example. ✓: the fold is proved; –: it is not.

Table 1 of Amy and Lunderville. The paper’s numbers, next to the tableau domain’s analyzed by hand with the joins of Section 8:

program	T in	PF _{Aff}	PF _{Pol}	tableau	reason for a gap
If-simple	2	0	0	0	
Loop-simple	2	0	0	0	
Loop-h	2	0	0	0	
Loop-swap	2	0	0	0	disjunctive join (per-row: 2)
Loop-null	4	2	2	2	
Grover (per iteration)	1736	1472	timeout	1472	
RUS	16	10	8	10	non-linear (Toffoli)
Loop-nonlinear	30	18	0	18	non-linear invariant
Loop-nested	3	2	2	3	reset: state fact
Reset-simple	2	1	1	2	reset: state fact

The tableau matches PF_{Aff} except on the two programs that need state facts from a reset. It trails PF_{Pol} on the two that need non-linear relations. Its advantages are elsewhere, in Clifford segments with H gates (Examples 11.2, 11.3 and 11.6), which Table 1 does not exercise.

Where the precision comes from.

- *Tableau*. It is exact on every Clifford, H included, and it proves X - and Y -type facts.
- *Relational analyses*.
 - *State facts*: constants after a reset, and ancillas known to be $|0\rangle$.
 - *Non-linear classical functions*: Toffoli gates and polynomial loop invariants.
 - *Interference found by path-sum rewriting*: the (H) rule on T -free variables.

The two kinds of analysis are sound independently, so their facts can be combined: a merge is valid if either analysis proves the segment fact. The two gaps in Table 1 are exactly the stabilizer extension and a non-linear component.

A Correspondence with the Lean formalization

Everything above is proved in Lean, except Proposition 6.1, its corollary, Lemma 6.3, the examples before Section 8, and Section 9 apart from Theorem 9.1, which are argued on paper only. The loop examples of Section 8 are checked in Lean by evaluation. The development is in the separate `lean-paulifold` repository, in the files

`PauliFold/Tableau.lean`, `Join.lean`, `Disjunctive.lean`, `FoldRule.lean`,

and its proofs are against the gate matrices of `TzapLean.gateUnitary`.

this note	Lean
Pauli strings $\mathcal{P}_n$, product, matrix	PP, PP.mul, PP.toMatrix
Lemma 1.2 (product)	PP.toMatrix_mul
Lemma 1.3 (commutation)	PP.Comm, PP.Comm.toMatrix
Lemma 1.4 (decomposition)	PP.restrict_cons, PP.restrict_all
Lemma 1.5 (rotations)	embed1_diag2, rotation_decomp
abstract states, σ_0	Tab, Tab.init
pull-back β_σ , Lemma 2.4	Tab.back, Tab.back_ok
transformers, abstract semantics	Tab.step, Tab.run
Table 2 (the transformer)	Tab.step
Lemma 3.2 (row updates)	Tab.step_rows, Tab.step_rows_back
right column of Table 2	pull_h_X, ..., pull_cz_Z, pullG_toMatrix
concretization γ	Tab.gamma
invariant, Lemmas 5.3–5.6	Tab.Inv, Tab.inv_init, Tab.inv_step, Tab.inv_run
Theorem 5.1	Tab.tableau_sound, Tab.circ_mem_gamma
partial states, completions	PTab, PTab.Extends (Join.lean)
Lemma 8.6	PTab.back_extends, gammaA_of_sound
Lemma 8.7	stepA_sound, sound_of_le, join_sound_left/right
programs, loops	Prog, Exec, loopFix, absRun
Theorem 8.8	absRun_sound, program_sound
loop examples	loopSimple_Z, loopH_Z, loopSwap
disjunctive states	DAbs, joinD (Disjunctive.lean)
Theorem 8.14	absRunD_sound, programD_sound
swap loop	loopSwap_ZZ
Theorem 9.1, the fact	segment_Z
Theorem 9.1, slide	fold_pos, fold_neg
Theorem 9.1, merge	fold_merge_pos, fold_merge_neg
α (their Def. 17), soundness, Prop. 21	alphaGammaC, alpha_sound, alphaGamma_mono
tableau vs. α	alpha_pauli_incomparable, midE_alpha_not_pauli
$\mathsf{T} m \mathsf{T} \rightarrow m \mathsf{S}$ (sign +), $\rightarrow m$ (sign –)	fold_t_t, fold_t_t_neg

In the Lean development:

- Commutation is *defined* as “ PQ and QP have the same phase”. By Lemma 1.3, this is equivalent to the parity condition.
- The Clifford transformer is the explicit update of Table 2. For Lemma 3.2, the right column is computed as $G^\dagger X_q G$ by the proved forward conjugation table of the inverse gate (pullG). Its form is stated gate by gate (pull_h_X, ...).
- The gate matrices are exactly those of Table 1.